11. Host Computer Comprehensive Control

- 11. Host Computer Comprehensive Control
    - 11.1 Experiment Objective
    - 11.2 Experiment Preparation
    - 11.3 Experiment Procedure
    - 11.4 Experiment Summary

11.1 Experiment Objective

This course focuses on learning how to generate host computer controls through a JupyterLab webpage to control the hexapod robot.

11.2 Experiment Preparation

Since the host computer control requires the use of a camera, to avoid conflicts, please close the APP control program or other programs that occupy the camera before running this program.

11.3 Experiment Procedure

Open the JupyterLab client and navigate to the code path:

```
muto/Samples/Control/11.Robot_Control.ipynb
```

By default, `g_ENABLE_CHINESE` is `False`. If you need to display Chinese, please set `g_ENABLE_CHINESE=True`.

```
g_ENABLE_CHINESE = True
Name_widgets = {
    'Stop': ("Stop", "Stop"),
    'Forward': ("Forward", "Forward"),
    'Backward': ("Backward", "Backward"),
    'Left': ("Left", "Translate Left"),
    'Right': ("Right", "Translate Right"),
    'TurnLeft': ("TurnLeft", "Turn Left"),
    'TurnRight': ("TurnRight", "Turn Right"),
    "Step":("Step", "Stride Width"),
    'Reset': ("Reset", "Reset to Initial Posture"),
    'Stretch': ("Stretch", "Stretch"),
    'Greeting': ("Greeting", "Greet"),
    'Retreat': ("Retreat", "Retreat in fear"),
    'Warm_up': ("Warm_up", "Warm-up squat"),
    'Turn_around': ("Turn_around", "Turn in place"),
    'Say_no': ("Say_no", "Wave to say no"),
    'Crouching': ("Crouching", "Curl up like a hermit crab"),
    'Stride': ("Stride", "Stride forward"),
    'Close_Camera': ("Close_Camera", "Close Camera")
}
```

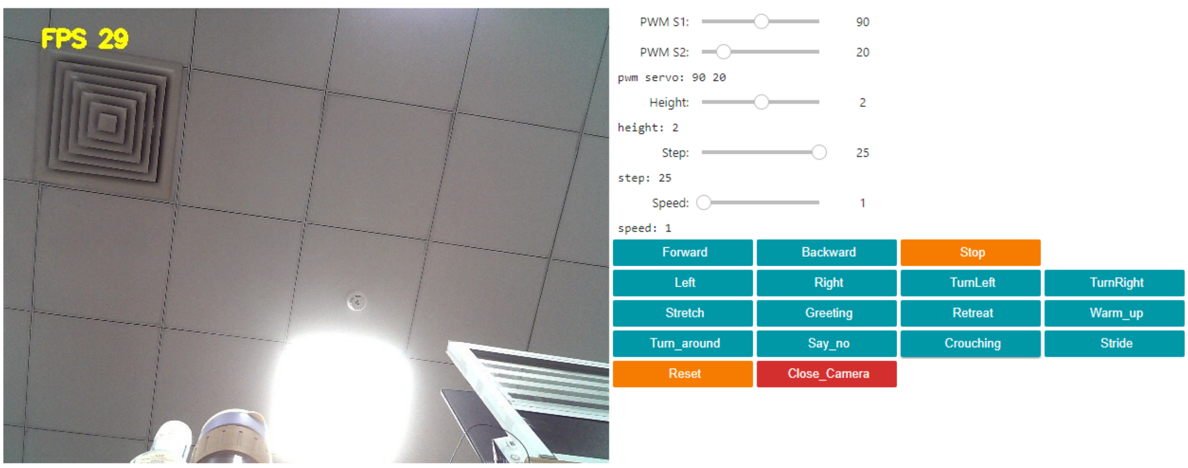
Turn on the camera. The default device is `/dev/video0`, with a display resolution of 640x480 and a frame rate of 30 fps. If the camera's device number in the system is not `/dev/video0`, please modify the configuration information according to the actual device number.

```
image_widget = widgets.Image(format='jpeg', width=640, height=480)
image = cv2.VideoCapture(0)
image.set(3, 640)
image.set(4, 480)
image.set(5, 30)
```

The camera display function reads the camera's feed and transmits the image to the `image_widget` control for display.

```
def Display_Camera():
    global g_stop, g_bot
    global g_height, g_width, g_roll, g_pitch, g_yaw
    t_start = time.time()
    fps = 0
    while not g_stop:
        ret, frame = image.read()
        fps = fps + 1
        mfps = fps / (time.time() - t_start)
        try:
            cv2.putText(frame, "FPS " + str(int(mfps)), (40,40),
cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,255,255), 2)
            image_widget.value = bgr8_to_jpeg(frame)
        except:
            pass
    image.release()
    g_bot.reset()
    time.sleep(.1)
    del g_bot
```

Click 'Run All Cells', then scroll to the bottom to see the generated controls.



On the left is the camera display, and on the right are the controls for the hexapod robot. The functions are the same as in the previous control sections, so they will not be explained further here.

The red button at the bottom is to close the camera process. When you want to end the program, please click 'Close Camera', otherwise it may keep the camera occupied and prevent other programs from using it.

11.4 Experiment Summary

In this experiment, JupyterLab controls are used to manage the hexapod robot's movement and actions. You can control the robot's movement or the camera gimbal to move the camera's view.

When finishing the program, you need to click the 'Close Camera' button, otherwise other programs using the camera will report an error.